Object Oriented Programming Quiz Question (C++)

Question 1 – Netrunners [10 Marks]

Netrunners are people who are able to communicate, modify and manipulate electronic smart devices scattered across Night City. To do this they use an arsenal of unique Data Structures.

A **NetRunner's Linked List** is a special type of linked list that keeps even and odd numbers separated within the same list. The list is divided into two sections:

- **Even Section:** Begins at the head of the list and contains all even numbers.
- **Odd Section:** Comes after the even section and contains all odd numbers.

Whenever a number is inserted into the list:

- If the number is **even**, it is inserted at the **beginning of the even section**.
- If the number is **odd**, it is inserted at the **beginning of the odd section** (i.e., immediately after the last even number).

For example:

Operation	Resulting List
Insert(5)	5
Insert(2)	$2 \rightarrow 5$
Insert(7)	$2 \rightarrow 7 \rightarrow 5$
Insert(19)	$2 \rightarrow 19 \rightarrow 7 \rightarrow 5$
Insert(22)	$22 \rightarrow 2 \rightarrow 19 \rightarrow 7 \rightarrow 5$

Lucyna Kushinada is one such Netrunner. Currently her `Insert` function adds every number to the head of the list regardless of the number being odd or even. Your task as a member of her crew is to make the required changes to the `Insert` function in `netrunner.cpp`.

Question 2 – Rotom in the City of Lights [15 Marks]

Rotom is a mischievous ghost type Pokémon who is able to take control of various electronic devices. Unfortunately, he has decided to take over the appliances in your home, which has caused a surge in your electricity bill (which has made K Electric VERY happy). To keep track of your electricity bill you've decided to create a few classes to represent these electronic devices and the energy they consume. Add these classes and their implementation to `rotom.cpp`.

Base Class: Appliance

Attributes:

- **Energy (float)** – total energy consumed (in Joules).
- **Duration (float)** – total time of operation (in seconds).
- **Status (boolean)** – indicates whether the appliance is turned on (`True`) or off (`False`).

Constructor:

- Initializes the attributes `Energy`, `Duration`, and `Status` with values provided as parameters.

Methods:

- `CalculateEnergy()` – Performs the operation `Energy / Duration` only if the appliance's `Status` is `True`. Otherwise, returns 0.
- `ToggleSwitch()` – Changes the value of `Status` from `True` to `False`, or from `False` to `True`.
- `GetStatus()` – Returns the current value of `Status`.

Derived Class 1: Fan

Additional Attribute:

- **Speed (string)** – represents the fan's speed setting. Can be "low", "medium", or "high".

Constructor:

- Inherits and calls the `Appliance` constructor to initialize `Energy`, `Duration`, and `Status`.
- Initializes `Speed` to "low" by default (unless otherwise specified).

Additional Methods:

- `SetSpeed(level)` – Sets the fan speed based on the parameter value ("low", "medium", or "high").
- `CalculateEnergy()` – Overrides the base class method.
  - If `Status` is `True`:
    - Returns `Energy / Duration + 50` if speed is "low",
    - `Energy / Duration + 75` if "medium",
    - `Energy / Duration + 100` if "high".
  - If `Status` is `False`, returns 0.

Derived Class 2: Heater

Additional Attribute:

- **TemperatureLevel (integer)** – represents the heating intensity, with valid values ranging from 1 to 5.

Constructor:

- Inherits and calls the `Appliance` constructor to initialize `Energy`, `Duration`, and `Status`.
- Initializes `TemperatureLevel` to 1 by default (lowest intensity).

Additional Methods:

- `SetTemperatureLevel(level)` – Sets the heating level to the specified value, ensuring it remains between 1 and 5.
- `CalculateEnergy()` – Overrides the base class method.
  - If `Status` is `True`, returns `(Energy / Duration) + (TemperatureLevel` $\times$ `30)`.
  - Otherwise, returns 0.

Derived Class 3: Refrigerator

Additional Attributes:

- **Temperature (float)** – represents the current internal temperature (in °C).
- **DoorOpen (boolean)** – indicates whether the refrigerator door is open.

Constructor:

- Inherits and calls the `Appliance` constructor to initialize `Energy`, `Duration`, and `Status`.
- Initializes `Temperature` to 4.0 (typical refrigerator temperature) and `DoorOpen` to `False` by default.

Additional Methods:

- `SetTemperature(temp)` – Sets the desired internal temperature to the specified value.
- `ToggleDoor()` – Changes the value of `DoorOpen` from `True` to `False`, or from `False` to `True`.
- `CalculateEnergy()` – Overrides the base class method.
  - If `Status` is `True`:
    - Base energy = `Energy / Duration`
    - Add +20 if `DoorOpen` is `True`.
    - Return the total.
  - If `Status` is `False`, return 0.

Question 3 – Final Fantasy [5 Marks]

In *Final Fantasy*, crystals are elemental objects which function as a source of magical energy.

Implement a class called `Crystal` that maintains a count which keeps track of the number of `Crystal` objects in existence globally. Each time a crystal is created, the counter is incremented, and each time a crystal is destroyed, the counter is decremented.

Instructions:

- Implement the `Crystal` class in `Crystal.cpp`.
- The main function is provided; **do not make any changes to it.**